1. Compare and Contrast CHAR vs VARCHAR

Char

- Fixed-length string datatype
- Always uses the full declared size
- Pads unused space with blanks
- Faster for consistently sized values

Varchar

- Variable-length string datatype
- Stores only actual entered characters
- Saves storage space
- Better for values with changing length

2. When to Use INTEGER or NUMERIC?

Integer

- Whole numbers only
- Faster calculations
- Less storage
  Use for: IDs, age, quantity

Numeric / Decimal

- Supports decimals
- More precise
- Best for exact values
  Use for: IDs, age, quantity

Why?

Use **INTEGER** when decimals are unnecessary

Use **NUMERIC** when precision matters, especially money

3. UNION vs UNION ALL

Union

- Combines results
- Removes duplicates
- Slower because it sorts/checks duplicates

Union all

- Combines results
- Keeps duplicates
- Faster because no duplicate filtering

4. Advantage of

a. Distinct

- Removes duplicate records
- Cleaner output
- Good for unique lists

Example:
SELECT DISTINCT city FROM customers;

b. Limit

- Restricts number of rows returned
- Faster response
- Useful for top records or previews

Example:
SELECT * FROM students LIMIT 10;

C. != Operator

- Means “not equal to”
- Filters unwanted values

Example:
WHERE age != 18

d. <> Operator

- Also means “not equal to”
- Standard SQL syntax
- More portable across systems

Example:
WHERE age <> 18

5.Discussion: How the EXPLAIN Command Optimizes SQL Queries

Based on the lesson’s focus on SQL optimization, the **EXPLAIN** command is one of the most useful tools for improving query performance because it shows how the database plans to execute a query before actually running it. It helps developers spot inefficiencies such as full table scans, missing indexes, or poor join order. SQL optimization guides consistently emphasize using **EXPLAIN** to identify where indexes should be added and whether the optimizer is choosing an efficient execution path

When you place EXPLAIN before a query, it displays:

- Which table is accessed first
- Whether indexes are used
- How many rows may be scanned
- Join order between tables
- Potential performance bottlenecks

EXPLAIN SELECT * FROM employees WHERE employee_id = 1001;

Why EXPALAIN is Important:

1.Detects Full Table Scans

If the database scans every row instead of using an index, the query becomes slower.

2. Shows Index Usage

EXPLAIN reveals whether your indexes are actually helping.

3.Improves JION Performance

It can show if tables are joined in an inefficient order.

4.Reduces Query Execution Time

By analyzing the plan, developers can rewrite queries for better speed.

Output Terms:

- **type** → How rows are searched (ALL = slow, index = faster)
- **possible_keys** → Available indexes
- **key** → Actual index used
- **rows** → Estimated rows scanned

Example Optimazation:

Inefficient

```
SELECT * FROM customers WHERE YEAR(created_at)=2024;
```

Better

```
SELECT * FROM customers WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';
```

The second query allows indexes on created_at to work properly, making it faster.

Main Advantages of EXPLAIN

- Finds slow queries
- Helps with indexing
- Improves database efficiency
- Prevents unnecessary scanning
- Supports better SQL design